

Support élève (Thème 1)



Activité 1 : Programmation fonctionnelle

Question 1. On souhaite sélectionner uniquement les élèves ayant obtenu au moins 10. Écrire une fonction `est_admis` qui reçoit un couple (`nom`, `note`) et renvoie :

- `True` si la note est supérieure ou égale à 10 ;
- `False` sinon.

Par exemple :

```
1      est_admis(("Alice", 15)) # doit renvoyer True
```

Utiliser ensuite la fonction `filter` pour sélectionner les élèves admis.

Question 2. On souhaite maintenant récupérer uniquement les noms des élèves admis. Écrire une fonction `recuperer_nom` qui reçoit un couple (`nom`, `note`) et renvoie le nom de l'élève.

Par exemple :

```
1      recuperer_nom(("Alice", 15)) # doit renvoyer "Alice"
```

Utiliser ensuite `map` pour appliquer cette fonction aux élèves admis.

On souhaite obtenir :

```
1      ['Alice', 'Chloé', 'David', 'Farid', 'Gabriel', 'Hugo', 'Inès']
```

Question 3. On souhaite maintenant récupérer uniquement les notes des élèves admis.

Écrire une fonction `recuperer_note` qui reçoit un couple (`nom`, `note`) et renvoie la note.

Par exemple :

```
1      recuperer_note(("Alice", 15)) # doit renvoyer 15
```

Utiliser `map` pour obtenir la liste des notes des élèves admis.

Résultat attendu :

```
1      [15, 12, 17, 14, 19, 10, 13]
```

Question 4. On souhaite maintenant calculer la somme des notes des élèves admis.

Importer la fonction `reduce` :

```
1 from functools import reduce
```

Écrire une fonction `additionner` qui reçoit deux nombres `x` et `y` et renvoie leur somme.

Utiliser `reduce` pour calculer la somme des notes obtenues à la question précédente.

Résultat attendu :

```
1 100
```

Question 5. On souhaite maintenant calculer la moyenne des élèves admis.

Utiliser :

- `filter` pour sélectionner les élèves admis ;
- `map` pour récupérer leurs notes ;
- `reduce` pour calculer leur somme ;
- `len` pour connaître leur nombre.

Le programme doit afficher :

```
1 Nombre d'admis : 7
2 Somme des notes : 100
3 Moyenne : 14.29
```



Activité débranchée 1 : Introduction

```

1  class Personne:
2      def __init__(self, lastName, firstName):
3          self.nom = lastName
4          self.prenom = firstName
5
6  eleve1 = Personne('NOM', 'Prenom') #à remplacer par les vôtres
7  eleve2 = Personne('NOM', 'Prenom') #à remplacer par les vôtres
8  eleve3 = Personne('NOM', 'Prenom') #à remplacer par les vôtres
9
10 class Chaise:
11     def __init__(self):
12         self.occupe = None
13         self.direction = 'N' #Nord
14
15     def occuper(self, eleve : Personne) -> None :
16         self.occupe = eleve
17
18     def liberer(self) -> None :
19         self.occupe = None
20
21     def est_occupee(self) -> bool :
22         return self.occupe is not None
23
24     def orienter(self, point_cardinal:str) -> None:
25         """oriente la chaise suivant les directions N (nord), S (sud), E (est), O (ouest)"""
26         self.direction = point_cardinal
27
28     def get_direction(self) -> str:
29         return self.direction
30

```

1. Numéroté les chaises puis matérialiser avec les chaises et les élèves de la classe l'application des instructions suivantes :

```

1  ch1 = Chaise()
2  ch2 = Chaise()
3  ch3 = Chaise()
4  ch2.occuper(eleve1)
5  ch2.orienter('E')

```

Faire vérifier par le professeur pour continuer.

```

1  ch3.occuper(eleve2)
2  ch3.orienter('S')
3  ch2.liberer()
4  ch2.orienter('O')

```

Faire vérifier par le professeur pour continuer.

2. A ce stade, que donnent les instructions suivantes ?

- a) ch1.est_occupee() ? et ch1.get_direction() ? et ch1.direction ?
- b) ch2.est_occupee() ?
- c) ch2.get_direction() ?
- d) ch3.est_occupee() ?
- e) ch3.direction ?

3. Pourquoi utilise-t-on des parenthèses pour `ch1.get_direction()` alors qu'on n'en utilise pas pour `ch1.direction` ?

```

1  class Pile_de_chaises :
2      def __init__(self):
3          self.pile = []
4      def empiler(self, chair:Chaise) -> None:
5          if chair.occupe == None :
6              if len(self.pile) > 0 and chair.direction == self.pile[-1].direction :
7                  self.pile.append(chair)
8              elif len(self.pile)==0 :
9                  self.pile.append(chair)
10
11     def diminuer_la_pile(self) -> None:
12         if len(self.pile)>1:
13             self.pile.pop()
14
15     def nb_chaises_empilees(self) -> int :
16         return len(self.pile)

```

4. Matérialiser avec les chaises et les élèves de la classe l'application des instructions suivantes :

<pre> 1 #Scénario 1 2 ch4=Chaise() 3 ch4.orienter('N') 4 5 ch5=Chaise() 6 ch5.orienter('O') 7 8 ch6=Chaise() 9 ch6.orienter('N') 10 11 ch7=Chaise() 12 ch7.occuper(eleve3) 13 14 ma_pile = Pile_de_chaises() 15 16 ma_pile.empiler(ch4) 17 ma_pile.empiler(ch5) 18 ma_pile.empiler(ch6) 19 ma_pile.empiler(ch7) 20 21 print(ma_pile.nb_chaises_empilees()) 22 23 24 "Faire vérifier par le professeur pour continuer." </pre>	<pre> #Scénario 2 (Suite du scénario 1) ch7.liberer() ch5.orienter('S') ch8=Chaise() ch8.orienter('S') ch8.occuper(eleve1) ma_pile.diminuer_la_pile() ma_pile.empiler(ch5) ma_pile.empiler(ch7) ma_pile.empiler(ch8) print(ma_pile.nb_chaises_empilees()) "Faire vérifier par le professeur." </pre>
--	---



Exercice 1 : C'est la classe

Soit le programme suivant :

```
1 class Ennemi:
2     def __init__(self):
3         self.point_de_vie = 20
4         self.difficulte_ia = "Facile"
5
6     def est_vaincu(self):
7         return self.point_de_vie <= 0
8
9     def degats_subis(self, degat):
10        self.point_de_vie -= degat
11
12    def mise_a_jour_ia(self):
13        if self.point_de_vie < 13:
14            self.difficulte_ia = "Moyenne"
15        elif self.point_de_vie < 5:
16            self.difficulte_ia = "Difficile"
17
18    marchand = Ennemi()
19
20    degat = 1
21    while not marchand.est_vaincu():
22        marchand.degats_subis(degat)
23        marchand.mise_a_jour_ia()
24        degat += 1
```

Question 1. Identifier le type de la variable marchand.

Question 2. Lister les attributs et méthodes de la classe Ennemi.

Question 3. Quelles sont les valeurs des attributs de l'objet marchand à sa création ?

Question 4. Noter à chaque tour de la boucle, les valeurs des attributs de marchand.



Activité 2 : Classe-ment

Question 1. Implémenter la classe `Ennemi` donnée précédemment. Puis lui ajouter un nouvel attribut nommé `arme`. Cet attribut correspond à un tuple (nom de l'arme, dégât de l'arme).

Question 2. Ajouter une méthode `prendre_arme` qui met à jour l'arme de l'Ennemi. Cette méthode prend en paramètres un nom d'arme et des dégâts.

Question 3. Ajouter ensuite une méthode permettant à la classe `Ennemi` de tirer nommée `faire_degat` et qui renvoie les dégâts de l'arme.

Question 4. Créer deux instances de la classe `Ennemi`. Puis effectuer un tirage aléatoire de valeurs, si la valeur est paire alors c'est le premier ennemi qui tire, réduisant la vie du second. Si la valeur est impaire c'est l'inverse. Réaliser des tirages aléatoires de valeurs jusqu'à la mort d'un des Ennemis et afficher le nombre de points de vie restant du vainqueur.



Activité 3 : Bot-aille

On récupère la classe `JeuDeCartes` suivante, les attributs ont été complétés, mais les méthodes non.

```
1 class JeuDeCartes:
2
3     def __init__(self):
4         """Construit un jeu de 52 cartes."""
5         self.cartes = []
6
7         self.valeurs = []
```

```
8         2, 3, 4, 5, 6, 7, 8, 9, 10,
9         "valet", "dame", "roi", "as"
10     ]
11
12     self.couleurs = [
13         "Pique", "Trèfle", "Carreau", "Coeur"
14     ]
15
16     # Création des 52 cartes
17     # À compléter (Question 1)
18
19     def nomCarte(self, c):
20         """Renvoie le nom d'une carte."""
21         # À compléter (Question 2)
22
23     def battre(self):
24         """Mélange les cartes."""
25         # À compléter (Question 3)
26
27     def tirer(self):
28         """Retire et renvoie une carte."""
29         # À compléter (Question 4)
30
31     # À compléter (Questions 5 et 6 )
```

On souhaite simuler le jeu de la bataille, utilisant 52 cartes. On sépare les cartes par ce qu'on appelle les couleurs (Trèfle, Carreau, Pique, Cœur). Chaque carte possède également une valeur (2, 3, 4, 5, 6, 7, 8, 9, 10, Valet, Dame, Roi, As).

Question 1. Compléter la méthode `init` de la classe `JeuDeCartes`, de façon à ce que l'attribut `cartes` contienne toutes les cartes du jeu. Autrement dit, que la liste contienne tous les tuples (valeur, couleur) possibles afin de représenter les cartes existantes dans un jeu de 52 cartes.

Question 2. Désormais, compléter la méthode `nomCarte` permettant d'afficher le nom d'une carte. (Ex : As de Pique)

Question 3. Compléter la méthode `battre` permettant de mélanger les cartes. Pour cela, utiliser la méthode `shuffle` de la bibliothèque `random`.

Question 4. Compléter la méthode `tirer` permettant de tirer la carte à l'indice 0 et de la supprimer de la liste de l'attribut `cartes`. La méthode retourne le tuple correspondant à la carte. Si toutes les cartes sont tirées, il faudra retourner `None`.

Question 5. Vérifier la méthode `tirer` en retirant toutes les cartes de la liste une par une avec une boucle `for`.

Question 6. Simuler un jeu de bataille entre deux bots (= joueur robot). Le joueur qui remporte le pli gagne un point. Vous avez le droit d'ajouter des méthodes si nécessaire.



Activité 4 : Appartement non meublé

On souhaite modéliser un appartement composé de plusieurs pièces. Une pièce est caractérisée par un nom et une surface en m². Un appartement est caractérisé par un nom et contient une liste de pièces. Compléter les méthodes des deux classes `Piece` et `Appartement`.

Question 1. Compléter la méthode `__init__` de la classe `Piece`. Elle doit permettre de mémoriser le nom et la surface de la pièce.

Question 2. Compléter les méthodes `getNom` et `getSurface` de la classe `Piece`.

Question 3. Compléter la méthode `setSurface` permettant de modifier la surface d'une pièce.

Question 4. Compléter la méthode `__init__` de la classe `Appartement`. Un nouvel appartement doit avoir un nom et une liste de pièces vide.

Question 5. Compléter la méthode `getNom` de la classe `Appartement`.

Question 6. Compléter la méthode `ajouter` permettant d'ajouter une pièce à l'appartement.

Question 7. Compléter la méthode `nbPieces` permettant de renvoyer le nombre de pièces de l'appartement.

Question 8. Compléter la méthode `getSurfaceTotale` permettant de calculer la surface totale de l'appartement.

Question 9. Compléter la méthode `getListePieces` permettant de retourner la liste des pièces de l'appartement.

Question 10. Créer un appartement et plusieurs pièces afin de tester les différentes méthodes.

Vérifier notamment que :

- les pièces sont correctement ajoutées à l'appartement ;
- le nombre de pièces est correct ;
- la surface totale est correctement calculée ;
- la modification de la surface d'une pièce est prise en compte.



Activité 5 : Temps pis

Créer et compléter une classe `Temps` permettant de représenter une durée à l'aide de trois attributs :

- `h` : le nombre d'heures ;
- `m` : le nombre de minutes ;
- `s` : le nombre de secondes.

La classe devra également comporter quatre méthodes :

- `__init__` : initialise les attributs de l'objet ;
- `add` : additionne deux durées et renvoie le résultat ;
- `sous` : soustrait une durée à une autre et renvoie le résultat ;
- `__repr__` : permet d'afficher une durée sous la forme « x heures y minutes z secondes ».

Question 1. Écrire la classe `Temps` et compléter la méthode `__init__`.

Question 2. Compléter la méthode `__repr__` permettant d'afficher une durée sous la forme : 2 heures 15 minutes 30 secondes

Question 3. Compléter la méthode `add` permettant d'additionner deux durées.
Par exemple :

```
1     t1 = Temps(2, 15, 30)
2     t2 = Temps(1, 50, 45)
3     t3 = t1.add(t2)
```

`t3` doit alors représenter une durée de 4 heures, 6 minutes et 15 secondes.

Question 4. Compléter la méthode `sous` permettant de soustraire une durée à une autre.
On supposera que la première durée est supérieure ou égale à la seconde.
Par exemple :

```
1     t1 = Temps(4, 30, 20)
2     t2 = Temps(1, 45, 50)
3     t3 = t1.sous(t2)
```

`t3` doit alors représenter une durée de 2 heures, 44 minutes et 30 secondes.



Exercice 2 : Ave César

Dans cet exercice, on étudie une méthode de chiffrement de chaînes de caractères alphabétiques appelée code de César. On considère que les messages à chiffrer sont composés uniquement de lettres majuscules de l'alphabet :

ABCDEFGHIJKLMNOPQRSTUVWXYZ

Le chiffrement utilise un nombre entier appelé clé de chiffrement. Cette clé détermine le décalage appliqué aux lettres du message. Soit la classe `CodeCesar` définie ci-dessous :

```
1     class CodeCesar:
2
3         def __init__(self, cle):
```

```

4         self.cle = cle
5         self.alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
6
7     def decale(self, lettre):
8         """Décale une lettre selon la clé de chiffrement."""
9
10        num1 = self.alphabet.find(lettre.upper())
11
12        # Si le caractère n'est pas une lettre de l'alphabet
13        if num1 == -1:
14            return lettre
15
16        # Application du décalage avec retour au début de l'alphabet
17        num2 = (num1 + self.cle) % 26
18
19        return self.alphabet[num2]
20

```

On rappelle que la méthode `str.find(lettre)` renvoie l'indice (index) de la lettre dans la chaîne de caractères `str`.

Question 1. Représenter le résultat d'exécution du code Python suivant :

```

1     code1 = CodeCesar(3)
2
3     print(code1.decale("A"))
4
5     print(code1.decale("X"))

```

La méthode de chiffrement du « code César » consiste à décaler les lettres du message dans l'alphabet d'un nombre de rangs fixé par la clé. Par exemple, avec la clé 3, toutes les lettres sont décalées de 3 rangs vers la droite : le A devient le D, le B devient le E, etc.

Question 2. Ajouter une méthode `chiffrement(self, texte)` dans la classe `CodeCesar` définie à la question précédente. Cette méthode reçoit en paramètre une chaîne de caractères correspondant au message à chiffrer et retourne le message chiffré. Elle doit utiliser la clé de chiffrement de l'instance et la méthode `decale`.

Exemple :

```

1     code1 = CodeCesar(3)
2     print(code1.chiffrement("NSI"))
3     # affiche "QVL"

```

Question 3. Écrire un programme qui :

- demande de saisir la clé de chiffrement ;

- crée un objet de classe `CodeCesar` avec cette clé ;
- demande de saisir le texte à chiffrer ;
- affiche le texte chiffré en appelant la méthode `chiffrement`.

Question 4. On ajoute la méthode `transforme(texte)` à la classe `CodeCesar` :

```
1     def transforme(self, texte):
2         self.cle = -self.cle
3         message = self.chiffrement(texte)
4         self.cle = -self.cle
5         return message
```

On exécute la ligne suivante :

```
1         code1 = CodeCesar(10)
2         print(code1.transforme("PSX"))
```

Que va-t-il se passer ? Expliquer votre réponse.



Activité 6 : Problème de colorisation

Un pays est composé de différentes régions. Deux régions sont voisines si elles ont au moins une frontière en commun. L'objectif est d'attribuer une couleur à chaque région de la carte sans que deux régions voisines aient la même couleur, en cherchant à utiliser un nombre de couleurs aussi faible que possible.

L'image ci-dessous (Figure 1) donne un exemple de résultat de colorisation des régions de la France métropolitaine.

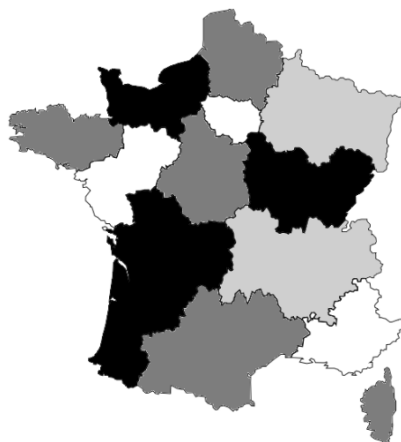


Figure 1 – Carte coloriée des régions de France métropolitaine

On rappelle quelques fonctions et méthodes des tableaux (le type `list` en Python) qui pourront être utilisées dans cet exercice :

- `len(tab)` : renvoie le nombre d'éléments du tableau `tab` ;
- `tab.append(elt)` : ajoute l'élément `elt` en fin de tableau `tab` ;
- `tab.remove(elt)` : enlève la première occurrence de `elt` de `tab` si `elt` est dans `tab`. Provoque une erreur sinon.

Partie 1

On considère la classe `Region` qui modélise une région d'un pays sur une carte.
Le début de son implémentation est donné ci-dessous :

```

1  class Region:
2      """Modélise une région d'un pays sur une carte."""
3
4      def __init__(self, nom_region):
5          """Initialise une région.
6
7              Paramètre :
8                  nom_region (str) : nom de la région.
9          """
10         self.nom = nom_region
11         self.tab_voisines = []
12         self.tab_couleurs_disponibles = ["rouge", "vert", "bleu", "jaune", "orange", "marron"]
13         self.couleur_attribuee = None
14

```

L'attribut `tab_voisines` contient la liste des régions voisines de la région.

L'attribut `tab_couleurs_disponibles` contient les couleurs qui peuvent encore être attribuées à la région.

Enfin, l'attribut `couleur_attribuee` contient la couleur attribuée à la région. Il vaut `None` lorsqu'aucune couleur ne lui a encore été attribuée.

Question 1. Donner une instruction permettant de créer une instance nommée `ge` de la classe `Region` correspondant à la région « Grand Est ».

Question 2. On souhaite ajouter à la classe `Region` une méthode `premiere_dispo` qui renvoie la première couleur disponible.

On supposera que la liste `tab_couleurs_disponibles` n'est pas vide.

```

1  def premiere_dispo(self):
2      """Renvoie la première couleur disponible.
3
4          Returns:
5              str : première couleur disponible.
6      """
7      return ...

```

Compléter la méthode ci-dessus.

Question 3. On souhaite ajouter à la classe `Region` une méthode `renvoie_nb_voisines` qui renvoie le nombre de régions voisines.

```

1  def renvoie_nb_voisines(self):
2      """Renvoie le nombre de régions voisines.
3
4      Returns:
5          int : nombre de régions voisines.
6      """
7      return ...

```

Compléter la méthode ci-dessus.

Question 4. On souhaite ajouter à la classe `Region` une méthode `est_colorisee` qui indique si une couleur a été attribuée à la région.

La méthode renvoie `True` si une couleur a été attribuée et `False` sinon.

```

1  def est_colorisee(self):
2      """Indique si une couleur a été attribuée à la région.
3
4      Returns:
5          bool : True si une couleur est attribuée, False sinon.
6      """
7      ...

```

Compléter la méthode ci-dessus.

Question 5. On souhaite ajouter à la classe `Region` une méthode `retirer_couleur`.

Cette méthode reçoit une couleur en paramètre et la retire de la liste des couleurs disponibles si elle s'y trouve.

Si la couleur n'est pas présente dans la liste, la méthode ne fait rien.

```

1  def retirer_couleur(self, couleur):
2      """Retire une couleur de la liste des couleurs disponibles.
3
4      Args:
5          couleur (str) : couleur à retirer.
6      """
7      ...

```

Compléter la méthode ci-dessus.

Question 6. On souhaite ajouter à la classe `Region` une méthode `est_voisine`.

Cette méthode reçoit une région en paramètre et renvoie `True` si cette région est une voisine de la région courante, et `False` sinon.

```

1  def est_voisine(self, region):
2      """Indique si une région est voisine de la région courante.
3
4      Args:
5          region (Region) : région à tester.
6
7      Returns:
8          bool : True si la région est voisine, False sinon.
9      """
10     ...

```

Compléter la méthode ci-dessus.

Partie 2

Dans cette partie :

- on dispose d'un ensemble d'instances de la classe `Region` pour lesquelles l'attribut `tab_voisines` a été renseigné;
- on pourra utiliser les méthodes de la classe `Region` écrites dans la partie 1 :

```

- premiere_dispo ;
- renvoie_nb_voisines ;
- est_colorisee ;
- retirer_couleur ;
- est_voisine.
```

On considère maintenant une classe `Pays` qui modélise la carte d'un pays composé de plusieurs régions.

Cette classe possède un unique attribut `tab_regions`. Il s'agit d'une liste Python dont les éléments sont des instances de la classe `Region`.

Question 7. On souhaite ajouter à la classe `Pays` une méthode `renvoie_tab_regions_non_coloriees` qui renvoie la liste des régions du pays auxquelles aucune couleur n'a encore été attribuée.

```

1  def renvoie_tab_regions_non_coloriees(self):
2      """Renvoie la liste des régions auxquelles aucune couleur n'a encore été attribuée.
3
4      Returns:
5      list : liste d'instances de la classe Region.
6      """
7      ...
```

Compléter la méthode ci-dessus.

Question 8. On considère la méthode suivante :

```

1  def renvoie_max(self):
2      """Renvoie une région non colorisée ayant le plus de voisines."""
3
4      nb_voisines_max = -1
5      region_max = None
6
7      for reg in self.renvie_tab_regions_non_coloriees():
8          if reg.renvie_nb_voisines() > nb_voisines_max:
9              nb_voisines_max = reg.renvie_nb_voisines()
10             region_max = reg
11      return region_max
```

1. Expliquer dans quel cas cette méthode renvoie `None`.
2. Dans le cas où cette méthode ne renvoie pas `None`, indiquer les deux particularités de la région renvoyée par rapport aux autres régions non colorisées.



Question 9. Écrire la méthode `colorie` de la classe `Pays` qui permet de coloriser toutes les régions du pays selon l'algorithme suivant :

- récupérer la région non colorisée qui possède le plus de voisines ;
- tant que cette région existe :
 - récupérer sa première couleur disponible ;
 - attribuer cette couleur à la région ;
 - pour chaque région voisine, retirer cette couleur de sa liste de couleurs disponibles si elle y est présente ;
 - récupérer à nouveau la région non colorisée qui possède le plus de voisines.

La méthode s'arrête lorsqu'il n'y a plus de région non colorisée.

On supposera qu'il reste toujours au moins une couleur disponible pour la région à coloriser.

Compléter la méthode suivante :

```
1  def colorie(self):
2      """Colorise toutes les régions du pays."""
3      ...
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```